

state 관리를 위한 입력, 출력, 동작 방식, 종료 조건 작성

승길 - 재료파악 에이전트

유승현 - 기분/상황 추천 에이전트

이승현 - 요리스타일 분기 에이전트

창학 - 가능 레시피 라우터 에이전트 : agentic ai 분기 판단 로직 들어가야 함

상재 - 레시피 생성 에이전트

최종 STATE 관리 변수 :

예시 :

```
import operator
from uuid import uuid4
from typing import Annotated, Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain_core.messages import BaseMessage

class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], operator.add]
    query: str
    query_analysis: dict
    search_results: list[dict]
    final_answer: str
    domain: str
    iteration_count: int

class QueryAnalysis(BaseModel):
    keywords: list[str] = Field(description="keywords")
    domain: Literal["medical", "general", "out_of_scope"] = Field(description="domain")
    intent: str = Field(description="intent")
    status: Literal["success", "insufficient"] = Field(description="status")

class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=2000)
    thread_id: str = Field(default_factory=lambda: str(uuid4()))

class ChatResponse(BaseModel):
    answer: str
    domain: str = ""
    sources: list[str] = Field(default_factory=list)
    disclaimer: str = ""
```

```
class StreamEvent(BaseModel):
    event: str = "message"
    node: str = ""
    data: str = ""
```

우리조거 작성 ㄱㄱ :

class AgentState(TypedDict):

#agent1

#agent2

#agent3

recipe_type: str

recipe_type_reason: str

#agent4

class RecipeRouterState(TypedDict, total=False):

LangGraph 메시지 누적용

messages: Annotated[list[BaseMessage], operator.add]

Input

available_ingredients: list[str]

ingredient_info: IngredientInfo

food_directions: FoodDirections

recipe_type: RecipeType

Output

candidate_foods: list[CandidateFood]

route: RecipeRoute

route_message: str

selected_recipe: Optional[SelectedRecipe]

ingredients_to_use: list[str]

seasonings_to_use: list[str]

substitutions: list[Substitution]

additional_ingredients: list[str]

#agent5

주제 :

멀티 에이전트 설계

전체 실행 흐름

사용자 입력

- 재료 파악 에이전트 및 기본·상황 추천 에이전트 병렬 실행
- 요리 스타일 분기 에이전트
- 가능 레시피 라우터 에이전트
- 레시피 생성 에이전트
- 최종 레시피 출력

1. 재료 파악 에이전트

1.1. 역할

사용자가 업로드한 식재료 사진을 분석하여 재료를 인식한다.

인식한 재료명을 표준화하고, 다른 에이전트가 활용할 수 있도록 주재료, 부재료, 양념류 및 영양 성격별로 분류한다.

1.2. 입력

- image_path
 - 사용자가 업로드한 식재료 사진의 경로
- image_id
 - 업로드된 이미지의 식별자
- user_input_ingredients
 - 사진 인식 결과를 보완하기 위해 사용자가 직접 입력하거나 수정한 재료 목록
- confidence_threshold
 - 인식 결과 중 사용자 확인이 필요한 재료를 분류하기 위한 신뢰도 기준값

1.3. 출력

- detected_ingredients
 - 사진에서 인식된 전체 식재료 후보 목록
 - 각 재료는 다음 정보를 포함한다.
 - name: 표준화된 식재료 이름
 - category: main, sub, seasoning 중 하나
 - nutrition_type: carbohydrate, protein, fat, vegetable, seasoning 중 하나
 - boundary_box: 이미지에서 해당 재료가 위치한 영역
 - confidence: 모델이 해당 재료로 판단한 신뢰도
 - needs_confirmation: 사용자 확인 필요 여부

- source: vision, manual, user_confirmed 중 하나
- uncertain_ingredients
 - 신뢰도가 낮아 사용자 확인이 필요한 식재료 후보 목록
- available_ingredients
 - 다음 에이전트가 실제로 사용할 수 있는 최종 식재료 목록
- ingredient_info
 - 식재료를 용도와 영양 성격에 따라 분류한 구조화 정보
 - 다음 항목을 포함한다.
 - main_ingredients: 요리의 중심이 되는 주재료 목록
 - sub_ingredients: 맛, 식감, 부피 등을 보조하는 부재료 목록
 - seasonings: 양념류 및 조미료 목록
 - carbohydrates: 탄수화물 재료 목록
 - proteins: 단백질 재료 목록
 - fats: 지방 또는 기름류 재료 목록
 - vegetables: 채소류 재료 목록
- vision_status
 - 재료 인식 결과 상태
 - success, need_user_confirmation, no_ingredient_detected, vision_error 중 하나
- vision_message
 - 재료 인식 결과에 대해 사용자에게 전달할 안내 메시지
- raw_vision_result
 - Vision 모델이 반환한 원본 인식 결과
 - 로그 확인 및 디버깅 용도로 활용

1.4. 동작 방식

1. image_path를 이용하여 Vision 모델로 식재료 후보를 탐지한다.
2. 탐지된 재료명을 표준 재료명으로 변환한다.
3. 각 재료를 main, sub, seasoning으로 분류한다.
4. 각 재료의 영양 성격을 carbohydrate, protein, fat, vegetable, seasoning 중 하나로 분류한다.
5. confidence_threshold 이상의 신뢰도를 가진 재료는 available_ingredients에 포함한다.
6. confidence_threshold보다 낮은 신뢰도를 가진 재료는 uncertain_ingredients에 포함한다.
7. uncertain_ingredients가 존재하면 사용자에게 확인을 요청한다.
8. 사용자가 직접 추가하거나 수정한 user_input_ingredients가 존재하면 결과에 반영한다.
9. 최종 재료 목록을 available_ingredients에 저장한다.
10. 분류된 재료 정보를 ingredient_info에 저장한다.
11. available_ingredients와 ingredient_info를 다음 에이전트에 전달한다.

1.5. 종료 조건

- success

- 정상적으로 인식된 식재료가 1개 이상 존재하는 경우
- available_ingredients와 ingredient_info가 정상적으로 생성된 경우
- 다음 에이전트 실행
- need_user_confirmation
 - 식재료 후보는 존재하지만 신뢰도가 낮은 항목이 포함된 경우
 - 사용자 확인 이후 재개
- no_ingredient_detected
 - 사진에서 식재료를 찾지 못한 경우
 - 사용자에게 사진 재업로드 또는 재료 직접 입력 요청
- vision_error
 - 이미지 경로 오류, 모델 호출 실패 또는 이미지 분석 실패가 발생한 경우
 - 오류 메시지 출력

2. 기본·상황 추천 에이전트

2.1. 역할

사용자가 입력한 현재 기분과 상황을 분석한다.

분석 결과를 바탕으로 조리 난이도, 선호 맛, 조리 방식, 권장 조리 시간 등 음식 추천 방향을 설정한다.

2.2. 입력

- user_mood_input
 - 사용자가 직접 입력한 현재 기분 또는 컨디션
 - 예시: 피곤하다, 스트레스를 받았다, 기분이 좋다
- user_situation_input
 - 사용자가 직접 입력한 현재 상황
 - 예시: 퇴근 직후, 늦은 밤, 시간이 부족함

2.3. 출력

- food_directions
 - 다음 에이전트가 공통으로 활용할 음식 추천 방향
 - 다음 항목을 포함한다.
 - mood: 표준화된 기분
 - situation: 표준화된 현재 상황
 - fatigue_level: 피로도
 - low, medium, high 중 하나
 - difficulty: 권장 조리 난이도
 - easy, normal, hard 중 하나
 - preferred_taste: 추천할 맛
 - 예시: spicy, mild, savory

- preferred_cooking_method: 추천할 조리 방식
 - 예시: stir_fry, boil, grill
- cooking_time_limit_minutes: 권장 최대 조리 시간

2.4. 동작 방식

1. user_mood_input에서 사용자의 기분과 컨디션을 분석한다.
2. user_situation_input에서 현재 상황과 조리 여건을 분석한다.
3. 사용자의 피로도를 low, medium, high 중 하나로 판단한다.
4. 사용자가 부담 없이 수행할 수 있는 조리 난이도를 설정한다.
5. 사용자의 기분과 상황에 맞는 맛과 조리 방식을 설정한다.
6. 적절한 최대 조리 시간을 설정한다.
7. 분석 결과를 food_directions에 저장한다.
8. food_directions를 다음 에이전트에 전달한다.

2.5. 종료 조건

- 정상 종료
 - food_directions가 정상적으로 생성된 경우
 - 다음 에이전트 실행
- 사용자 입력 없음
 - user_mood_input 또는 user_situation_input이 비어 있는 경우
 - 기본 추천 방향을 설정하고 다음 에이전트 실행
- 분석 실패
 - 사용자 입력 분석에 실패한 경우
 - 기본 추천 방향을 설정하고 오류 내용을 로그에 기록

3. 요리 스타일 분기 에이전트

3.1. 역할

재료 정보와 사용자의 기분·상황 정보를 종합하여 가장 적합한 요리 스타일을 선택한다.

지원하는 요리 스타일은 한식, 중식, 일식, 양식이다.

3.2. 입력

- ingredient_info
 - 재료 파악 에이전트가 생성한 구조화된 식재료 정보
 - 주재료, 부재료, 양념류 및 영양 성격별 분류 정보 포함
- food_directions
 - 기분·상황 추천 에이전트가 생성한 음식 추천 방향
 - 기분, 상황, 피로도, 조리 난이도, 선호 맛, 선호 조리 방식, 최대 조리 시간 포함

3.3. 출력

- recipe_type
 - 최종 선택된 요리 스타일
 - korean, chinese, japanese, western 중 하나
- recipe_type_reason
 - 해당 요리 스타일을 선택한 이유

3.4. 동작 방식

1. ingredient_info에서 주재료, 부재료, 양념류를 확인한다.
2. 각 재료가 한식, 중식, 일식, 양식 중 어떤 스타일과 잘 어울리는지 평가한다.
3. food_directions에서 사용자의 기분, 상황, 피로도, 선호 맛, 선호 조리 방식 및 조리 시간을 확인한다.
4. 각 요리 스타일이 사용자의 현재 상황에 적합한지 평가한다.
5. 한식, 중식, 일식, 양식별 적합도 점수를 계산한다.
6. 가장 높은 점수를 받은 요리 스타일을 recipe_type에 저장한다.
7. 해당 요리 스타일을 선택한 근거를 recipe_type_reason에 저장한다.
8. recipe_type과 recipe_type_reason을 다음 에이전트에 전달한다.

3.5. 종료 조건

- 정상 종료
 - korean, chinese, japanese, western 중 하나의 recipe_type이 선택된 경우
 - 다음 에이전트 실행
- 판단 불가
 - 입력 데이터가 부족하여 특정 스타일을 선택하기 어려운 경우
 - recipe_type = null, recipe_type_reason = null로 저장
 - 기본 스타일을 적용하거나 사용자에게 추가 입력 요청

4. 가능 레시피 라우터 에이전트

4.1. 역할

현재 보유한 재료와 사용자 조건을 바탕으로 후보 음식을 생성한다.

각 후보 음식이 현재 재료로 실제 조리 가능한지 판단하고, 가장 적절한 음식을 선택한다.

부족한 재료가 있다면 대체, 생략 또는 최소 추가 구매 여부를 결정한다.

4.2. 입력

- available_ingredients
 - 재료 파악 에이전트가 생성한 실제 사용 가능한 재료 목록
- ingredient_info
 - 재료 파악 에이전트가 생성한 구조화된 재료 정보

- food_directions
 - 기분·상황 추천 에이전트가 생성한 음식 추천 방향
- recipe_type
 - 요리 스타일 분기 에이전트가 선택한 요리 스타일

4.3. 출력

- candidate_foods
 - 조건에 맞는 후보 음식 목록
- route
 - 레시피 생성 진행 방식을 결정하는 분기 상태
 - can_cook, simple_recipe, missing_ingredient, constraint_conflict 중 하나
- route_message
 - 분기 결과에 대해 사용자에게 전달할 안내 메시지
- selected_recipe
 - 최종적으로 선택한 음식 정보
- ingredients_to_use
 - 실제 레시피 생성에 사용할 일반 재료 목록
- seasonings_to_use
 - 실제 레시피 생성에 사용할 양념 목록
- substitutions
 - 대체하거나 생략할 수 있는 재료 목록
 - 각 항목은 다음 정보를 포함한다.
 - original: 원래 필요한 재료
 - replacement: 대체 재료
 - reason: 대체하거나 생략할 수 있는 이유
 - 재료를 생략하는 경우 replacement는 null로 설정할 수 있다.
- additional_ingredients
 - 조리를 위해 추가로 준비해야 하는 최소 재료 목록

4.4. 동작 방식

1. recipe_type에 적합한 후보 음식을 생성한다.
2. 각 후보 음식의 필수 재료와 available_ingredients를 비교한다.
3. food_directions를 확인하여 사용자의 조리 시간, 난이도, 선호 맛 및 조리 방식에 적합한지 판단한다.
4. 현재 조건에 가장 적합한 음식을 selected_recipe에 저장한다.
5. 실제 사용할 일반 재료를 ingredients_to_use에 저장한다.
6. 실제 사용할 양념을 seasonings_to_use에 저장한다.
7. 대체하거나 생략할 수 있는 재료를 substitutions에 저장한다.
8. 추가로 필요한 최소 재료를 additional_ingredients에 저장한다.
9. 현재 재료와 사용자 조건을 바탕으로 route를 결정한다.

10. route와 관련된 안내 문구를 route_message에 저장한다.
11. 레시피 생성이 가능한 경우 결과를 레시피 생성 에이전트에 전달한다.

4.5. 종료 조건

- can_cook
 - 현재 보유한 재료만으로 바로 조리 가능한 경우
 - 레시피 생성 에이전트 실행
- simple_recipe
 - 일부 재료를 대체하거나 생략하면 조리 가능한 경우
 - 레시피 생성 에이전트 실행
- missing_ingredient
 - 핵심 재료가 부족하여 현재 상태로 조리하기 어려운 경우
 - 사용자에게 추가로 필요한 최소 재료 안내
- constraint_conflict
 - 사용자의 조리 시간, 난이도 또는 선호 조건과 재료가 충돌하는 경우
 - 사용자에게 조건 변경 요청 또는 다른 후보 음식 탐색

5. 레시피 생성 에이전트

5.1. 역할

앞 단계에서 선택된 음식과 확정된 재료를 바탕으로 사용자가 직접 따라 할 수 있는 최종 레시피를 생성한다. 요리 초보자도 쉽게 이해할 수 있도록 재료의 양, 불의 세기, 조리 시간 및 조리 순서를 구체적으로 안내한다.

5.2. 입력

- selected_recipe
 - 가능 레시피 라우터 에이전트가 선택한 최종 음식
- ingredients_to_use
 - 실제 레시피에 사용할 일반 재료 목록
- seasonings_to_use
 - 실제 레시피에 사용할 양념 목록
- substitutions
 - 대체하거나 생략할 재료 목록
- additional_ingredients
 - 추가로 준비해야 하는 최소 재료 목록
- servings
 - 기준 인분
- food_directions
 - 사용자의 기분과 상황을 반영한 음식 추천 방향
 - 조리 난이도, 선호 조리 방식 및 최대 조리 시간 포함

5.3. 출력

- generated_recipe
 - 사용자에게 최종 제공할 완성된 레시피
 - 다음 항목을 포함한다.
 - recipe_name: 최종 요리 이름
 - ingredients: 실제 사용할 재료와 분량
 - cooking_steps: 단계별 조리 순서
 - cooking_time_minutes: 예상 조리 시간
 - difficulty: 조리 난이도
 - servings: 기준 인분
 - cooking_tips: 추가 조리 팁
 - substitutions: 대체하거나 생략한 재료 안내
 - additional_ingredients: 추가로 필요한 최소 재료 안내
- generation_status
 - 레시피 생성 결과
 - success, failed 중 하나
- generation_message
 - 레시피 생성 결과에 대해 사용자에게 전달할 안내 메시지

5.4. 동작 방식

1. selected_recipe와 확정된 재료 목록을 전달받는다.
2. ingredients_to_use와 seasonings_to_use를 기준으로 레시피를 생성한다.
3. substitutions가 존재하면 대체하거나 생략할 재료를 반영한다.
4. additional_ingredients가 존재하면 별도로 안내한다.
5. servings를 기준으로 각 재료의 분량을 계산한다.
6. food_directions를 확인하여 조리 난이도와 조리 시간을 조절한다.
7. 사용자가 쉽게 따라 할 수 있도록 단계별 조리 순서를 작성한다.
8. 필요한 경우 불의 세기, 조리 시간 및 주의 사항을 구체적으로 작성한다.
9. 추가 조리 팁을 작성한다.
10. 최종 결과를 generated_recipe에 저장한다.

5.5. 종료 조건

- success
 - 다음 필수 항목이 모두 정상적으로 생성된 경우
 - recipe_name
 - ingredients
 - cooking_steps
 - cooking_time_minutes

- difficulty
 - servings
- 최종 레시피를 사용자에게 출력
- failed
 - selected_recipe가 존재하지 않는 경우
 - 필수 항목 생성에 실패한 경우
 - 오류 메시지 출력

6. 에이전트 간 State 변수 연결 요약

재료 파악 에이전트

→ available_ingredients, ingredient_info

→ 요리 스타일 분기 에이전트 및 가능 레시피 라우터 에이전트

기분·상황 추천 에이전트

→ food_directions

→ 요리 스타일 분기 에이전트, 가능 레시피 라우터 에이전트 및 레시피 생성 에이전트

요리 스타일 분기 에이전트

→ recipe_type, recipe_type_reason

→ 가능 레시피 라우터 에이전트

가능 레시피 라우터 에이전트

→ selected_recipe, ingredients_to_use, seasonings_to_use, substitutions, additional_ingredients

→ 레시피 생성 에이전트

레시피 생성 에이전트

→ generated_recipe, generation_status, generation_message

→ 사용자 화면